

DMML Assignment

MLflow Experiment Tracking with DagsHub Centralized Collaboration

Group Members Name with Student ID:

1. Bhooshaan A - 2025cs05032
2. Shanmathy S - 2025cs05019
3. Sathiya L - 2025cs05023
4. Swaminathan M - 2025cs05009

Recording Link:

<https://drive.google.com/file/d/1o9aU2Y9pKBKLu3LeSSSR1boyFNB9eKAm/view?usp=sharing>

Repository Link:

<https://github.com/Bhooshaan-2025cs05032/DMML-Assignment>

Architecture Diagram:

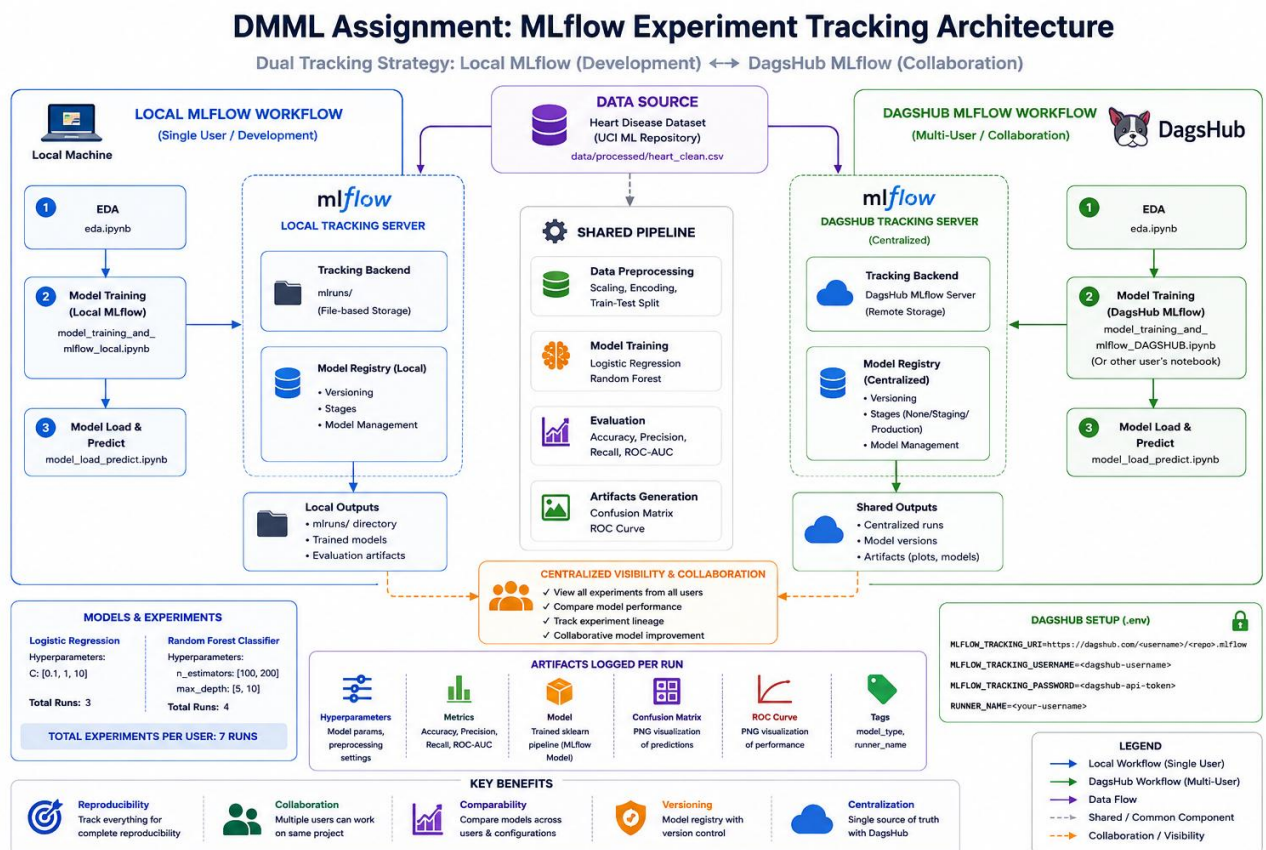


Figure 1: MLflow + DagsHub Experiment Tracking Architecture

1. Project Overview

This assignment demonstrates the implementation of MLflow-based experiment tracking and DagsHub centralized MLflow integration for collaborative machine learning workflows. The project focuses on training and evaluating heart disease prediction models while tracking hyperparameters, metrics, artifacts, and model versions in both local and remote MLflow environments.

2. Objectives

- Implement experiment tracking using MLflow.
- Enable centralized collaboration using DagsHub MLflow.
- Track model parameters, metrics, and artifacts.
- Maintain reproducibility of experiments.
- Register and version trained machine learning models.
- Compare multiple machine learning configurations and performance.

3. Dataset Information

The project uses the Heart Disease dataset obtained from the UCI Machine Learning Repository. The dataset contains patient health attributes such as age, cholesterol, blood pressure, and other medical indicators used to predict the presence or absence of heart disease.

Attribute	Details
Dataset Source	UCI Machine Learning Repository
Problem Type	Binary Classification
Target Variable	Heart Disease Presence
Preprocessing	Scaling, Encoding, Train-Test Split

4. Folder Structure

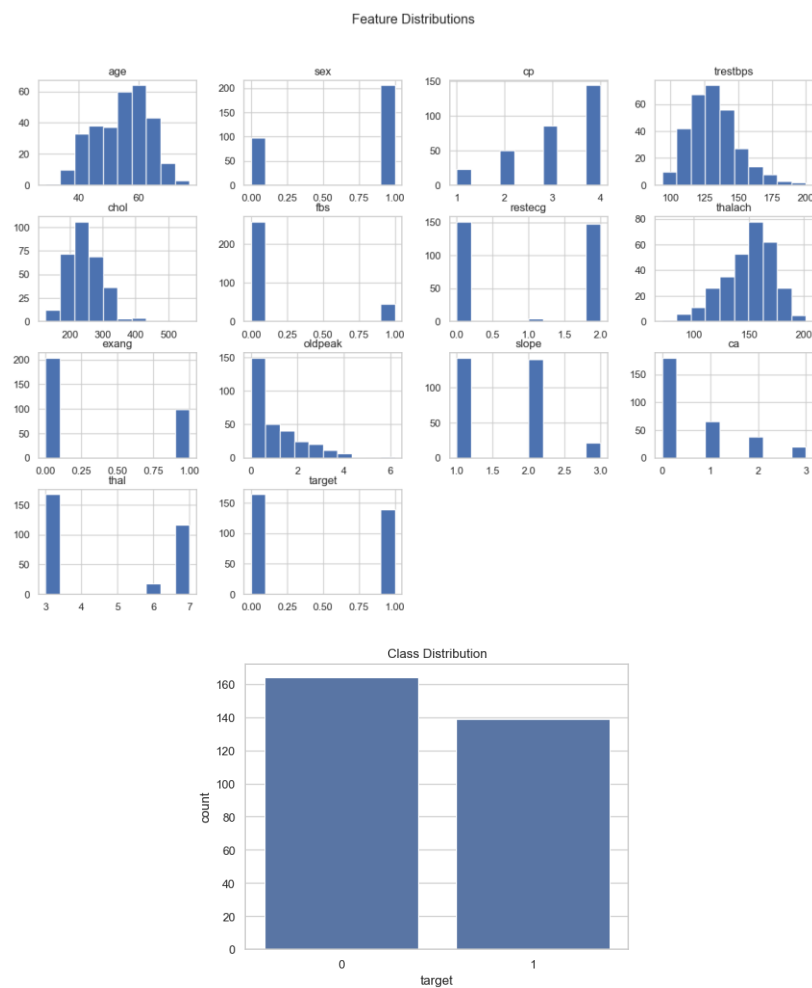
```
DMML-Assignment/  
|  
├── README.md  
├── DOCUMENTATION.md  
├── notebooks/  
|   ├── eda.ipynb  
|   ├── model_training_and_mlflow_local.ipynb  
|   ├── model_training_and_mlflow_DAGSHUB.ipynb  
|   └── model_load_predict.ipynb  
└── data/
```

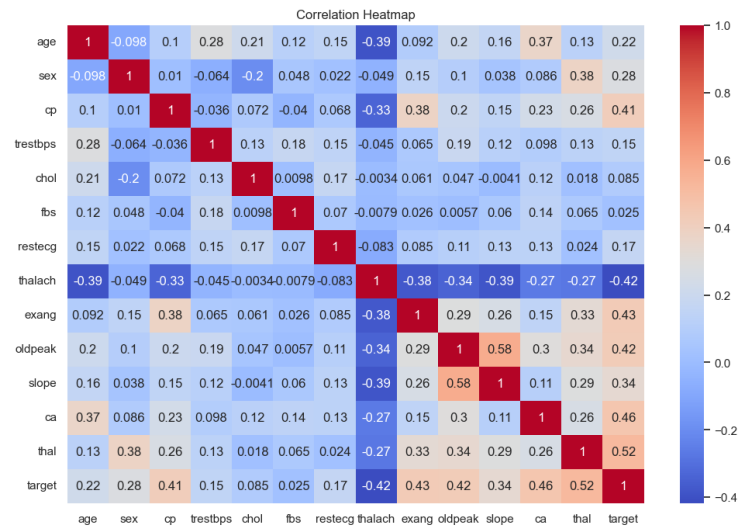
└─ artifacts/
└─ models/
└─ mlruns/
└─ dmmmlenv/

5. Workflow Explanation

5.1 Exploratory Data Analysis (EDA)

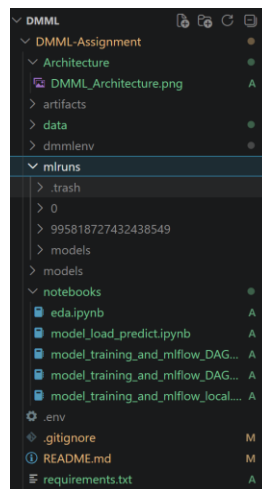
The EDA notebook performs dataset inspection, feature analysis, missing value handling, and visualization of correlations and feature distributions.



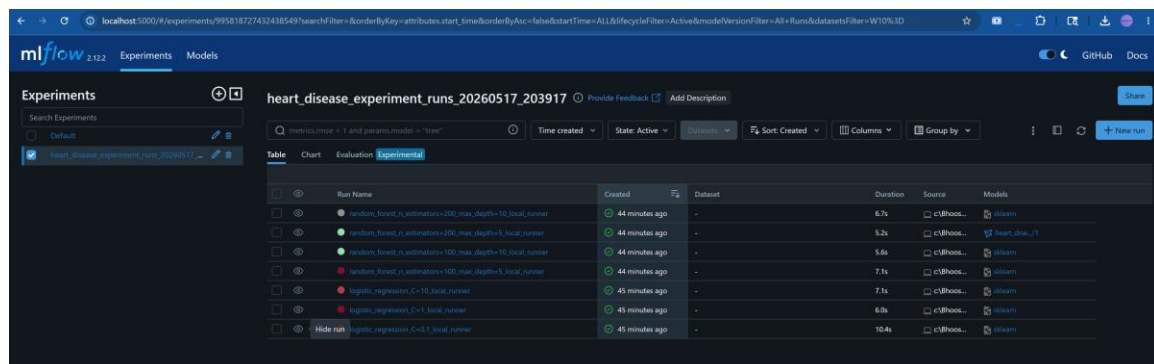


5.2 Local MLflow Tracking

Local MLflow tracking is used during development and debugging. All experiment runs are stored in the local mlruns directory.



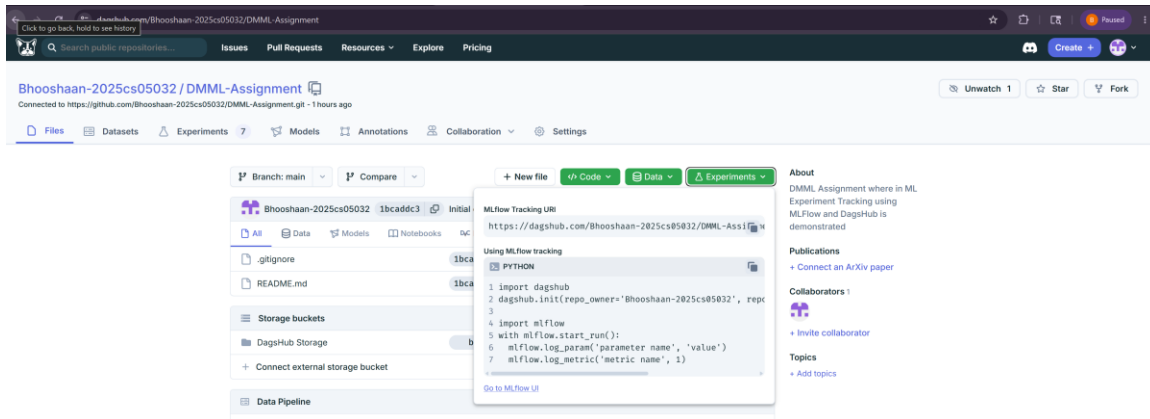
mlruns Folder in Local Directory



mlflow running on localhost:5000

5.3 DagsHub MLflow Tracking

DagsHub acts as a centralized MLflow tracking server enabling multi-user collaboration. All users can log experiments to a shared backend and compare results.



DagsHub Repository Linked to Github Repository with Centralized MLFlow URL

5.4 Model Loading and Prediction

The final trained model can be loaded either from the MLflow Model Registry or from a locally stored pickle file for inference.

```
model = load_model()

print("Model type:", type(model))
```

✓ 5.4s

```
2026-05-17 20:37:09,047 - INFO - Trying MLflow Registry...
MLFLOW_TRACKING_URI from .env: https://dagshub.com/Bhooshaan-2025cs05032/DMML-Assignment.mlflow
Using DagsHub MLflow URI: https://dagshub.com/Bhooshaan-2025cs05032/DMML-Assignment.mlflow
2026-05-17 20:37:14,471 - INFO - Loaded from MLflow
Model type: <class 'sklearn.pipeline.Pipeline'>
```

```
healthy_sample = {
    "age": 40,    "sex": 0,    "cp": 1,          # less severe chest pain
    "trestbps": 110, "chol": 180, "fbs": 0,    "restecg": 0,    "thalach": 170,    # high max heart rate (good sign)
    "exang": 0,    "oldpeak": 0.0,    # no ST depression
    "slope": 2,    "ca": 0,    "thal": 2
}

disease_sample = {
    "age": 65,    "sex": 1,    "cp": 4,          # severe chest pain
    "trestbps": 160, "chol": 300, "fbs": 1,    "restecg": 2,    "thalach": 100,    # low max heart rate
    "exang": 1,    "oldpeak": 4.0,    # strong ST depression
    "slope": 0,    "ca": 3,    "thal": 3
}

print("Healthy Sample Prediction:")
prediction = predict_heart_disease(healthy_sample).get("prediction")
confidence = predict_heart_disease(healthy_sample).get("confidence")
print({
    "prediction": int(prediction),
    "confidence": float(round(confidence, 4))
})

print("Disease Sample Prediction:")
prediction = predict_heart_disease(disease_sample).get("prediction")
confidence = predict_heart_disease(disease_sample).get("confidence")
print({
    "prediction": int(prediction),
    "confidence": float(round(confidence, 4))
})

✓ 0.1s

2026-05-17 20:37:22,172 - INFO - Prediction result: {'prediction': 0, 'confidence': 0.8302}
2026-05-17 20:37:22,202 - INFO - Prediction result: {'prediction': 0, 'confidence': 0.8302}
2026-05-17 20:37:22,234 - INFO - Prediction result: {'prediction': 1, 'confidence': 0.872}
2026-05-17 20:37:22,268 - INFO - Prediction result: {'prediction': 1, 'confidence': 0.872}
Healthy Sample Prediction:
{'prediction': 0, 'confidence': 0.8302}
Disease Sample Prediction:
{'prediction': 1, 'confidence': 0.872}
```

6. Models Evaluated

Model	Hyperparameters	Total Runs
Logistic Regression	C = [0.1, 1, 10]	3
Random Forest	n_estimators=[100,200], max_depth=[5,10]	4

7. Experiment Tracking Components

- Hyperparameters: Model configuration values.
- Metrics: Accuracy, Precision, Recall, ROC-AUC.
- Artifacts: Confusion Matrix, ROC Curve, Trained Models.
- Tags: Runner Name and Model Type.
- Model Registry: Version control and deployment readiness.

8. DagsHub Integration

DagsHub provides a centralized MLflow tracking backend that enables collaboration between multiple users. Using DagsHub integration, experiments, metrics, artifacts, and model versions are accessible through a common dashboard.

DagsHub URL: <https://dagshub.com/Bhooshaan-2025cs05032/DMML-Assignment.mlflow>

MLFLOW_TRACKING_URI=https://dagshub.com/<username>/<repo>.mlflow

MLFLOW_TRACKING_USERNAME=<username>

MLFLOW_TRACKING_PASSWORD=<api-token>

RUNNER_NAME=<your-name>

9. Technology Stack

Library	Version	Purpose
pandas	2.2.3	Data Analysis
numpy	1.26.4	Numerical Computing
scikit-learn	1.5.2	Machine Learning
mlflow	2.12.2	Experiment Tracking
matplotlib	3.10.3	Visualization
python-dotenv	1.0.1	Environment Variables

10. Key Features Demonstrated

- Hyperparameter Optimization
- Experiment Reproducibility
- Model Registry and Versioning
- Artifact Management
- Collaborative Experiment Tracking
- Evaluation Metric Logging
- Centralized Visibility using DagsHub

11. Learning Outcomes

- Understanding MLflow experiment tracking.
- Working with centralized MLflow backends.
- Logging metrics, artifacts, and parameters.
- Collaborative machine learning workflows.
- Managing model versions using MLflow Registry.

12. Conclusion

This project successfully demonstrates a complete experiment tracking workflow using MLflow and DagsHub. The implementation highlights best practices for reproducible

machine learning experimentation, centralized collaboration, model versioning, and artifact management.